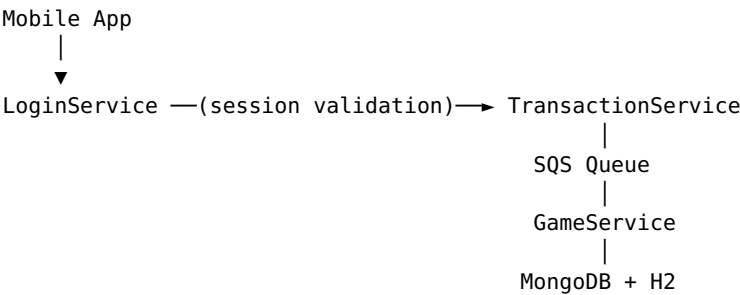


Relatório de Arquitetura — Itaú Microservices Platform

Data: 27 de Abril de 2026
Repositório: ThiagoCintra/itau-microservices-infra
Branch: feature/arquivos

1. Visão Geral

A plataforma é composta por três microsserviços independentes, uma camada de infraestrutura gerenciada via Docker Compose e comunicação assíncrona via SQS (LocalStack em ambiente local). A arquitetura garante isolamento de domínios, tolerância a falhas e processamento com garantia de exatamente-uma-vez (*exactly-once*).



2. Serviços

2.1 LoginService (porta 8081)

Responsabilidade: Gateway de autenticação e gerenciamento de sessões.

Tecnologia	Papel
Spring Boot 4.0.5	Framework da aplicação
Spring Security	Pipeline de autenticação e filtro JWT
Spring Data Redis	Armazenamento de sessões e rate limiting
JJWT 0.11.5	Geração e validação de JWT
H2 Database	Persistência de usuários (local)
Java 21 Virtual Threads	Alta concorrência

Endpoints:

Método	Caminho	Descrição
POST	/api/v1/login	Autentica o usuário e emite JWT
GET	/api/v1/me	Retorna a sessão ativa para um JWT

Mecanismos de segurança: - Rate limiting via Lua INCR no Redis (máx. 5 tentativas/IP/60s) - JWT HS256 com customerId, channel, sessionId e role - Sessões armazenadas no Redis com TTL de 5 minutos

2.2 TransactionService (porta 8080)

Responsabilidade: Aceitar e validar transações financeiras, publicar eventos no SQS.

Fluxo de uma requisição:

1. Validação da assinatura JWT (local, sem chamada de rede)
2. Chamada síncrona para LoginService GET /me — confirma sessão ativa e contractService == true
3. Validação de negócio (channel == MOBILE, tipo/valor válidos)
4. Publicação do TransactionEvent na fila SQS transactions
5. Retorno HTTP 202 Accepted

Resiliência na chamada ao LoginService: - Retry: 3 tentativas, backoff exponencial, base 500ms (Resilience4j) - Circuit Breaker: abre após 50% de falhas em 10 chamadas, fica aberto por 30s

2.3 GameService (porta 8082)

Responsabilidade: Processar eventos de transação de forma assíncrona e aplicar regras de gamificação.

Fluxo de processamento:

1. Poll SQS (long-polling, até 20s, até 10 mensagens/batch)
2. Cada mensagem processada em uma virtual thread independente
3. Verificação de idempotência no MongoDB (GameEventDocument por eventId)
4. Registro do evento no MongoDB (idempotência camada 1)
5. Filtro: apenas transações PIX são elegíveis
6. Reset mensal de pontos se necessário (CustomerProgress.lastReset)
7. Verificação se benefício já foi resgatado no mês
8. Verificação de idempotência no H2 (ProcessedEvent por eventId) — camada 2
9. Avaliação de missões (Mission.minValue ≤ amount ≤ Mission.maxValue)
10. Recálculo de nível (LevelRule.minPoints ≤ totalPoints)
11. Salvamento com optimistic locking (@Version)
12. Registro do ProcessedEvent no H2
13. Delete do SQS message (acknowledge explícito)

Em falha: visibilidade estendida exponencialmente (60s × receiveCount, máx. 12h); após 5 falhas → DLQ.

3. Infraestrutura

Componente	Tecnologia	Papel
Fila principal	SQS (LocalStack)	transactions — buffer de eventos
Fila de falhas	SQS (LocalStack)	transactions-dlq — mensagens não processadas
Cache / sessões	Redis	Rate limiting e sessões JWT
Banco de gamificação	MongoDB	Eventos e progresso do

Banco de usuários/regras	H2 (JPA)	cliente Usuários, missões, níveis, eventos processados
--------------------------	----------	---

4. Garantia de Idempotência

O sistema garante processamento **exactly-once** mesmo com a semântica *at-least-once* do SQS:

Camada	Mecanismo	Banco	Cobre
Camada de mensagem	Lookup do GameEventDocument por eventId	MongoDB	Reentregas antes de qualquer estado escrito
Camada de lógica de negócio	Registro ProcessedEvent antes de aplicar regras	H2 (JPA)	Reentregas após escrita no MongoDB mas antes do commit no H2

5. Fluxo Completo

Fase	Serviço	Ação	Propriedade Arquitetural
Autenticação	LoginService	Emitir JWT + criar sessão	Gateway único de auth
Aceitação da transação	TransactionService	Validar + publicar evento	Isolamento do caminho crítico
Validação de sessão	LoginService	Confirmar sessão ativa	Segurança sem estado armazenado no TS
Buffering do evento	SQS	Persistir evento durável	Desacoplamento + retry
Gamificação	GameService	Processar regras de forma assíncrona	Isolamento de falhas + escala independente

6. Decisões de Design

Decisão	Justificativa
JWT com validação local em TransactionService	Evita latência — não requer chamada de rede para parsing de identidade
Única chamada	Confirmação de sessão ativa é pré-requisito de segurança; não pode ser

síncrona (GET /me)	adiada
SQS para gamificação	Gamificação é downstream e não afeta o resultado da transação; falhas são isoladas
Dupla idempotência (MongoDB + H2)	SQS at-least-once exige exactly-once na aplicação; duas camadas cobrem todos os cenários de falha
Virtual Threads (Java 21)	Alto throughput de I/O (Redis, MongoDB, SQS) sem overhead de thread pool clássico
Redis para sessões	Lookups de sessão estão no hot path — sub-milissegundo é obrigatório

7. Conclusão

A arquitetura entrega isolamento de domínio, resiliência em cascata e garantia de processamento correto sob falhas parciais. Cada serviço pode ser escalado, reiniciado ou substituído de forma independente sem impactar o fluxo principal de transações.